

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code Repository

The complete implementation of this experiment, including dataset exploration, data preprocessing, baseline MLP construction, model training, performance evaluation, hyperparameter optimization using `RandomizedSearchCV`, has been made available through the following GitHub repository.

GitHub Repository:

<https://github.com/Siddharth-G06/Deep-Learning-Lab/tree/main/Lab%202%20-%20Multi-Layer%20Perceptron>

The repository enables reproducibility of the experiment and allows the implementation to be executed, verified, and extended using the same methodology described in this report.

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **`GridSearchCV`** or **`RandomizedSearchCV`** (recommended) together with the `SciKeras` wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

9. Plots with Inference

Generate the following plots.

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch
5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix
8. Hyperparameter Search Results
9. Best Model Accuracy Comparison

Write a 2–3 line inference for each plot.

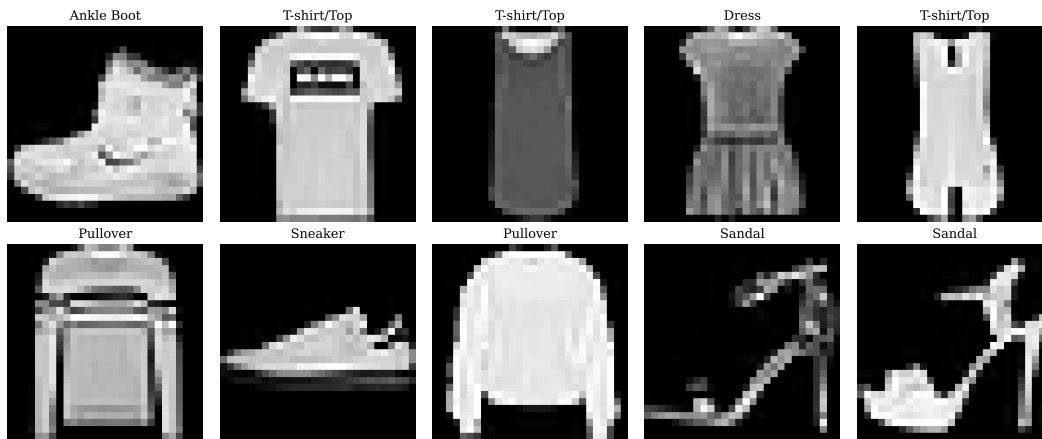


Figure 1: Sample Images from the Fashion-MNIST Dataset

Inference:

The sample images demonstrate the diversity of clothing categories available in the Fashion-MNIST dataset. Each grayscale image belongs to one of the ten predefined classes and serves as the input for the multi-class classification model.

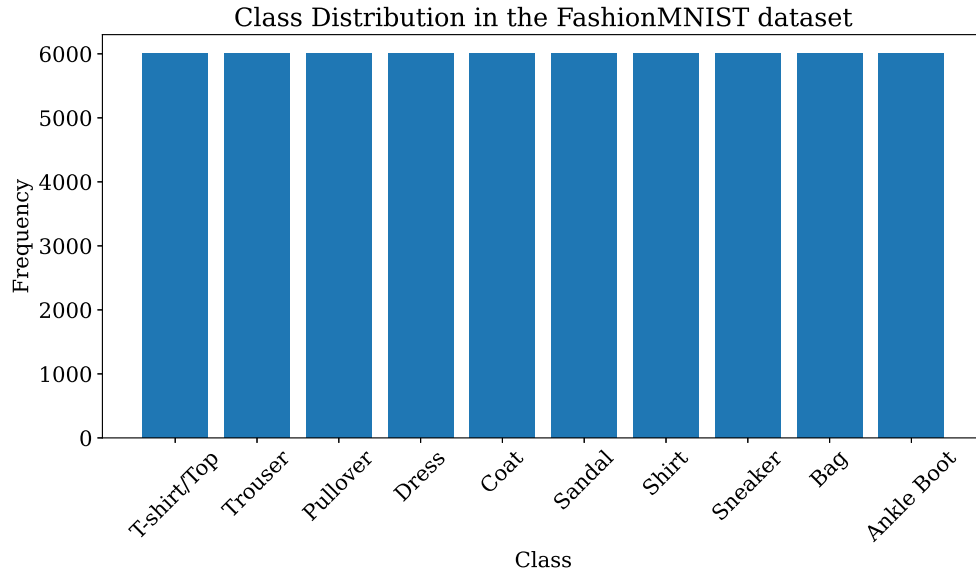


Figure 2: Class Distribution of the Fashion-MNIST Dataset

Inference:

The class distribution is uniform, with approximately 6,000 training images in each category. This balanced dataset minimizes class imbalance and allows the MLP to learn each category without bias.

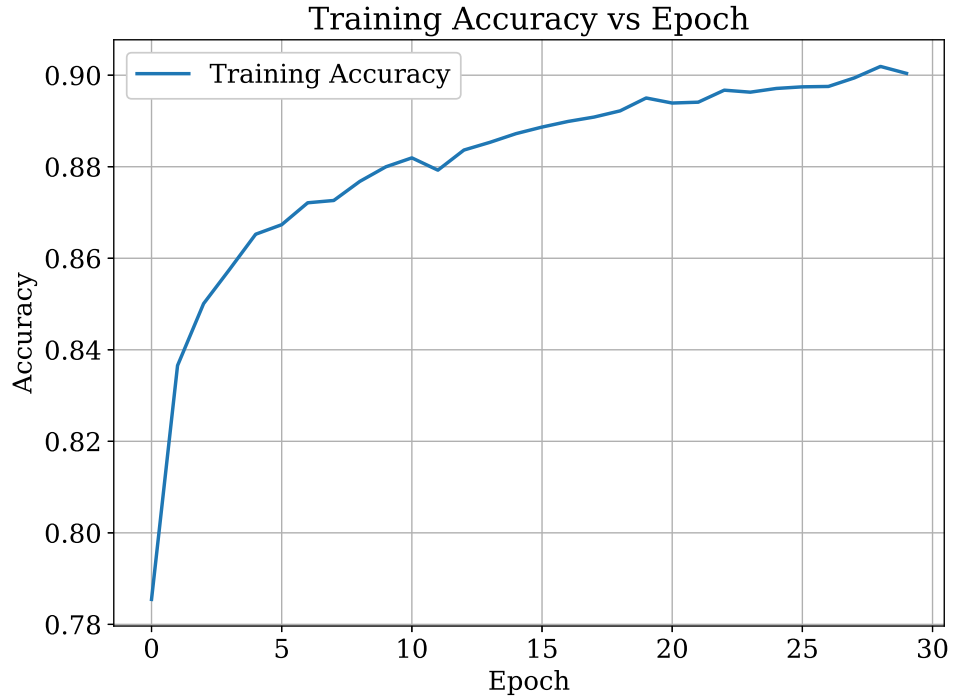


Figure 3: Training Accuracy versus Epoch

Inference:

The training accuracy increased steadily over successive epochs before converging. This indicates that the model had successfully learned meaningful feature representations from the training dataset.

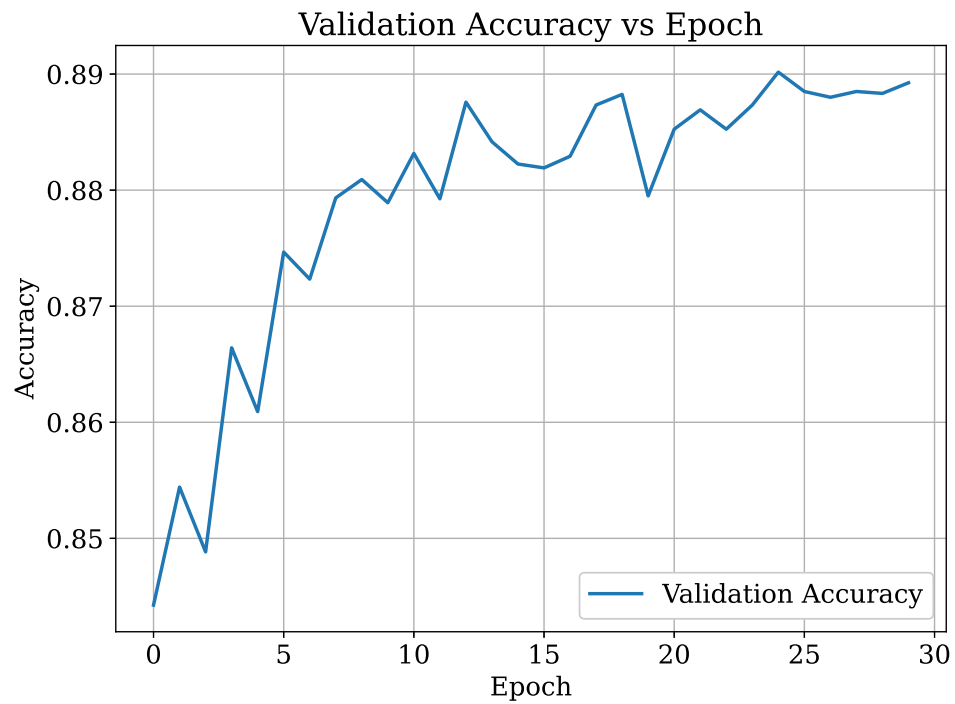


Figure 4: Validation Accuracy versus Epoch

Inference:

The validation accuracy followed a trend similar to the training accuracy and stabilized after several epochs. The small gap between the training and validation curves indicates good generalization with limited overfitting.

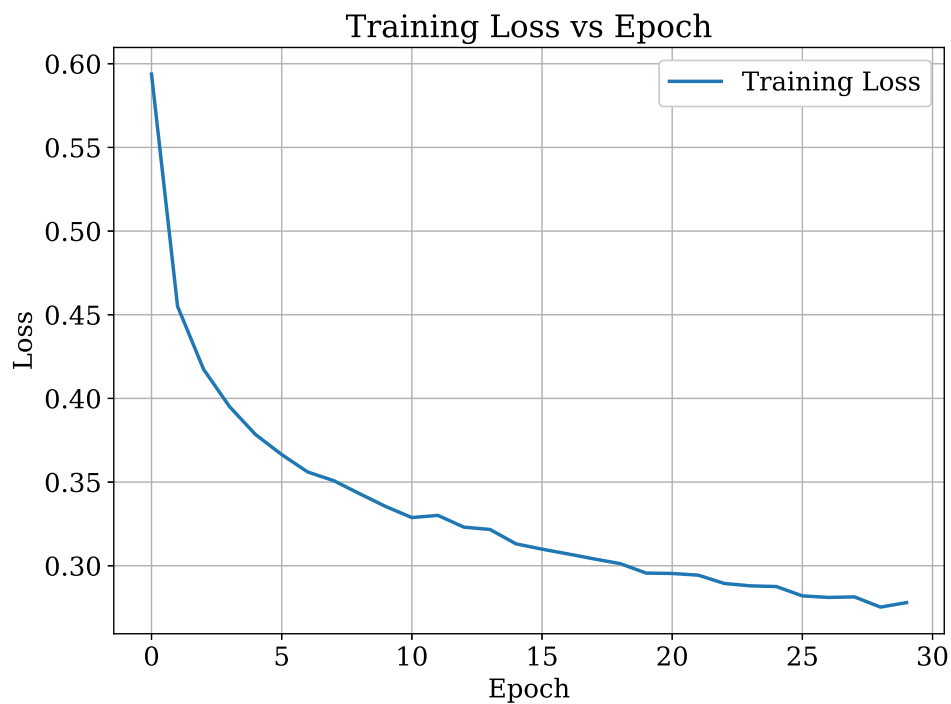


Figure 5: Training Loss versus Epoch

Inference:

The training loss decreased consistently throughout the learning process, indicating effective optimization of the model parameters and the convergence towards a lower error.

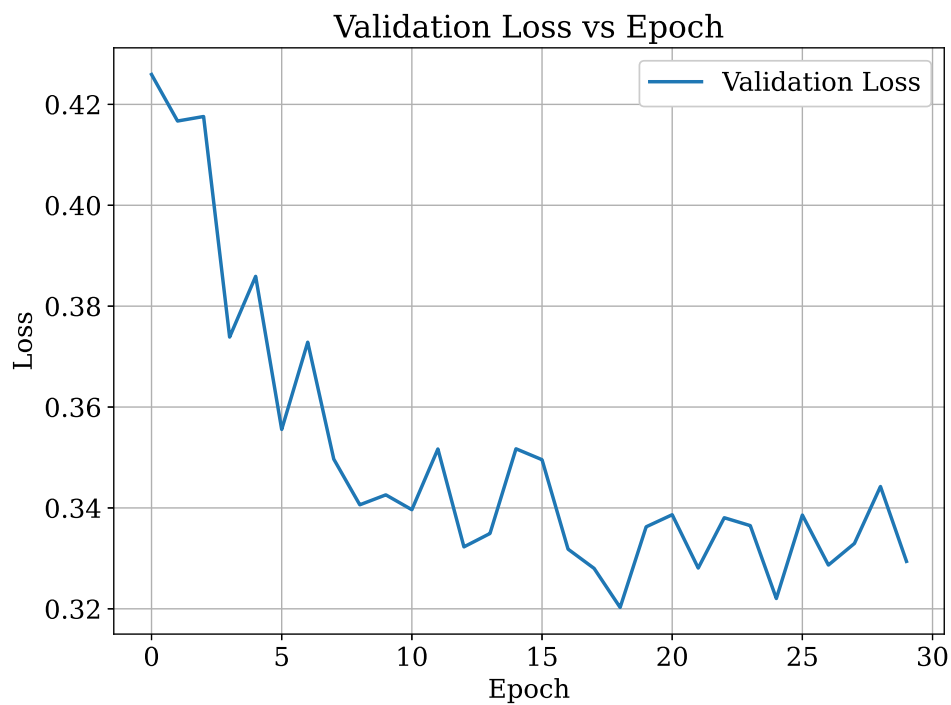


Figure 6: Validation Loss versus Epoch

Inference:

The validation loss has decreased rapidly during the initial epochs and later it stabilized. The absence of a significant increase in the validation loss suggests that the model had generalized well on unseen data.

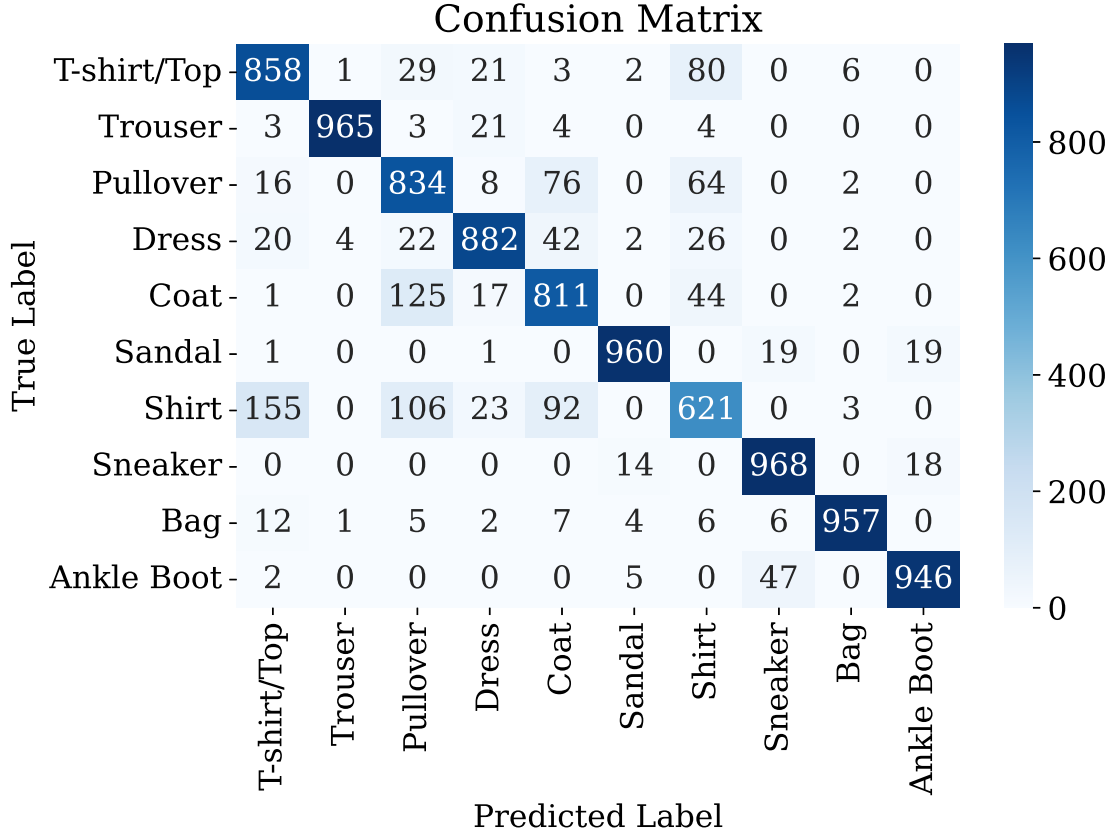


Figure 7: Confusion Matrix of the Baseline MLP

Inference:

Most predictions lie along the main diagonal, indicating correct classification for the majority of test samples. Misclassifications primarily occur between visually similar categories such as Shirt, Pullover, and Coat, which share overlapping visual characteristics.

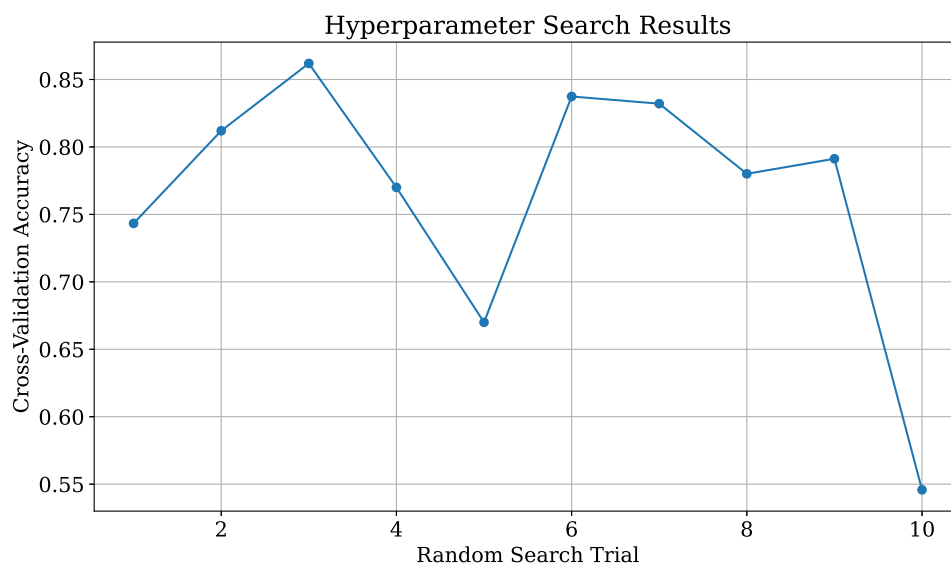


Figure 8: Hyperparameter Search Performance

Inference:

Different hyperparameter combinations produced varying cross-validation accuracies during Randomized Search. The search successfully identified a competitive configuration for retraining the optimized MLP model.

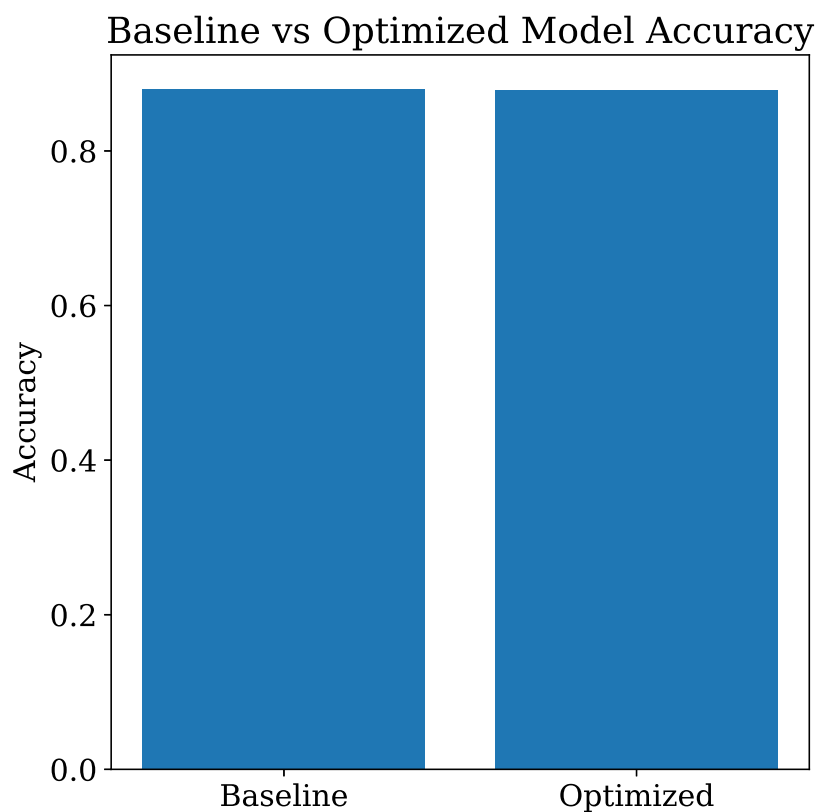


Figure 9: Comparison of Baseline and Optimized MLP Accuracy

Inference:

The optimized model achieved performance comparable to the baseline model but did not exceed its testing accuracy. This indicates that the baseline architecture was already well suited for the Fashion-MNIST dataset, while the optimized configuration maintained similar generalization performance. A likely factor might be using the subset of 12,000 training samples to reduce computational time.

10. Results

Table 1. Hyperparameters

Hyperparameter	Best Value
Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	rmsprop
Activation Function	tanh
Epochs	30
Dropout Rate	0.2
Cross Validation Accuracy	0.8619
Testing Accuracy	0.8790

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.880200	0.879000
Precision	0.881263	0.878630
Recall	0.880200	0.879000
F1-score	0.879768	0.877712
Training Time	2 min (approx)	30 min (approx)

11. Discussion

1. Which optimization method was used?

RandomizedSearchCV together with the SciKeras wrapper was used to perform automated hyperparameter optimization. A 5-fold cross-validation strategy had been adopted to evaluate the different combinations of the hyperparameters and to identify a suitable MLP configuration.

2. Which hyperparameters were selected?

The optimized model was obtained by tuning the number of hidden layers, number of hidden neurons, the learning rate, optimizer, activation function, dropout rate, batch size, and the number of training epochs. The final values selected by RandomizedSearchCV are presented in the Results section.

3. Did optimization improve performance?

The optimized model achieved performance comparable to the baseline model but did not outperform it on the testing dataset. A likely contributing factor is that the hyperparameter search

was performed using a reduced subset of 12,000 training samples to reduce computational time. Consequently, the selected hyperparameters may not represent the optimal configuration for the complete Fashion-MNIST dataset. However, the optimized model demonstrated similar generalization performance to the baseline model.

4. Which hyperparameter had the greatest impact?

Among the tuned parameters, the learning rate and optimizer had the greatest influence on the model's convergence and overall classification performance. The network architecture, including the number of hidden layers and neurons, also affected the model's learning capacity and generalization.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible combination of hyperparameters, making it computationally expensive for large search spaces. In contrast, Randomized Search evaluates only a randomly selected subset of combinations, significantly reducing computation time while still identifying competitive hyperparameter configurations. Therefore, Randomized Search is more suitable for deep learning models with numerous tunable parameters.

6. Recommend the best MLP configuration.

For this experiment, the baseline MLP architecture consisting of two hidden layers with 128 and 64 neurons, ReLU activation, the Adam optimizer, categorical cross-entropy loss, and a batch size of 32 is recommended. Although Randomized Search produced a competitive model, the baseline configuration achieved marginally better testing accuracy while maintaining a simpler architecture.

12. Conclusion

In this experiment, a Multi-Layer Perceptron (MLP) was successfully implemented for multi-class image classification using the Fashion-MNIST dataset. The dataset was preprocessed by flattening the 28×28 grayscale images into 784-dimensional feature vectors, normalizing the pixel values to the range $[0, 1]$, and converting the class labels into one-hot encoded vectors suitable for categorical classification. A baseline MLP architecture consisting of two hidden layers with ReLU activation and a Softmax output layer was developed using TensorFlow/Keras and trained using the Adam optimizer with categorical cross-entropy loss.

The trained model demonstrated strong classification performance on the Fashion-MNIST test dataset, achieving high accuracy, precision, recall, and F1-score. The confusion matrix indicated that the majority of test images were classified correctly, while a small number of misclassifications occurred between visually similar clothing categories such as shirts, pullovers, and coats. The training and validation accuracy and loss curves showed stable convergence, indicating that the network learned meaningful representations without significant overfitting.

Automated hyperparameter optimization was then performed using RandomizedSearchCV with the SciKeras wrapper to explore different combinations of hidden layers, hidden neurons, learning rates, optimizers, activation functions, dropout rates, batch sizes, and training epochs. Although the optimized model achieved performance comparable to the baseline model, it did not produce a higher testing accuracy. A likely reason is that the hyperparameter search was conducted on a reduced subset of 12,000 training samples to reduce computational time, which may have limited the ability of the search process to identify the globally optimal configuration for the complete dataset.

Overall, this experiment provided practical experience in the complete deep learning workflow, including data preprocessing, neural network design, model training, performance evaluation, and hyperparameter optimization. It also demonstrated that while automated hyperparameter optimization is a powerful technique for improving model performance, a well-designed baseline architecture can already achieve highly competitive results on balanced benchmark datasets such as Fashion-MNIST. The exper-

iment highlights the importance of selecting appropriate network architectures, optimization strategies, and evaluation metrics when developing deep learning models for image classification tasks.

13. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

Additional Exercise

1 Objective

The objective of this experiment is to implement and analyze a Multi-Layer Perceptron (MLP) for solving the XOR classification problem. The experiment demonstrates the limitation of a single-layer perceptron on non-linearly separable data and illustrates how the introduction of a hidden layer enables successful learning. The model is implemented using TensorFlow and evaluated using training accuracy, loss, and prediction results.

2 Experimental Procedure

The experiment consists of two parts. In the first part, a single-layer perceptron is implemented from scratch to classify the XOR dataset. Since XOR is not linearly separable, the perceptron fails to converge even after multiple training epochs.

In the second part, a Multi-Layer Perceptron (MLP) is implemented using TensorFlow. The network consists of one hidden layer with eight neurons using the hyperbolic tangent (tanh) activation function and an output neuron with the sigmoid activation function. The model is trained using the Adam optimizer with a learning rate of 0.05 for 1000 epochs. The training process is analyzed using accuracy and loss curves, and the final predictions are evaluated.

3 Network Architecture

The architecture of the implemented Multi-Layer Perceptron is summarized in Table 1.

Table 1: MLP Architecture

Input Layer	2 Neurons
Hidden Layer	8 Neurons (tanh)
Output Layer	1 Neuron (sigmoid)
Optimizer	Adam
Learning Rate	0.05
Loss Function	Binary Crossentropy
Epochs	1000

4 Results

The single-layer perceptron was unable to classify the XOR dataset because the classes are not linearly separable. The training error did not converge to zero even after the maximum number of epochs.

The Multi-Layer Perceptron successfully learned the XOR function and achieved perfect classification of all four input combinations. The training accuracy approached 100%, while the binary crossentropy loss decreased steadily during training.

5 Results

The single-layer perceptron was unable to classify the XOR dataset because the classes are not linearly separable. The training error did not converge to zero even after the maximum number of epochs.

The Multi-Layer Perceptron successfully learned the XOR function and achieved perfect classification of all four input combinations. The training accuracy approached 100%, while the binary crossentropy loss decreased steadily during training.

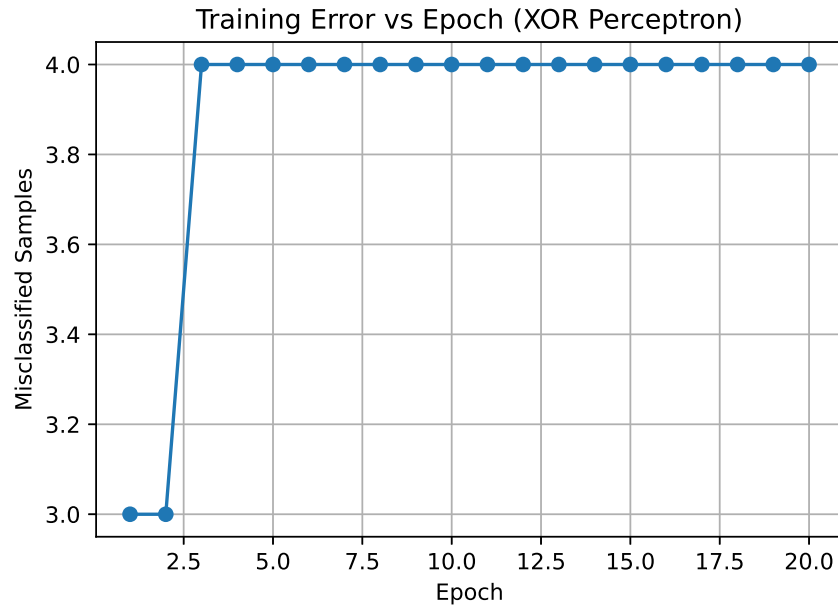


Figure 10: Training error versus epoch for the single-layer perceptron on the XOR dataset.

Inference: The single-layer perceptron fails to solve the XOR problem because the classes are not

linearly separable. The training error does not converge to zero even after multiple epochs.

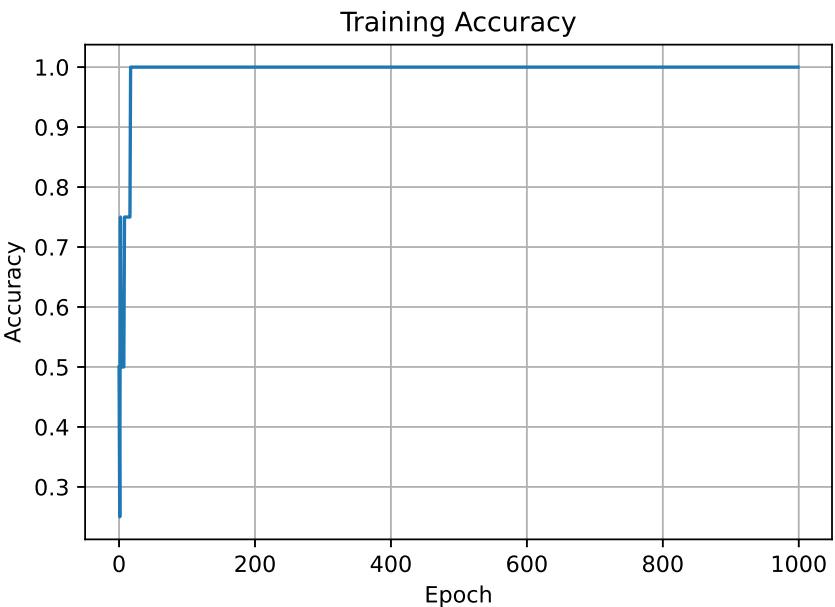


Figure 11: Training accuracy of the Multi-Layer Perceptron.

Inference: The training accuracy gradually increases and reaches 100%, indicating that the Multi-Layer Perceptron successfully learns the XOR function.

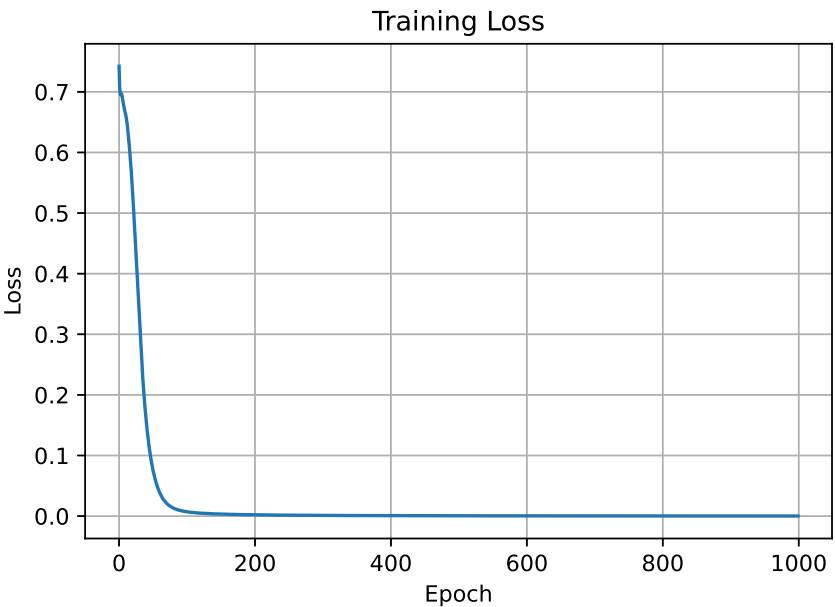


Figure 12: Training loss of the Multi-Layer Perceptron.

Inference: The binary crossentropy loss decreases steadily throughout training, demonstrating effective optimization and successful convergence of the neural network.

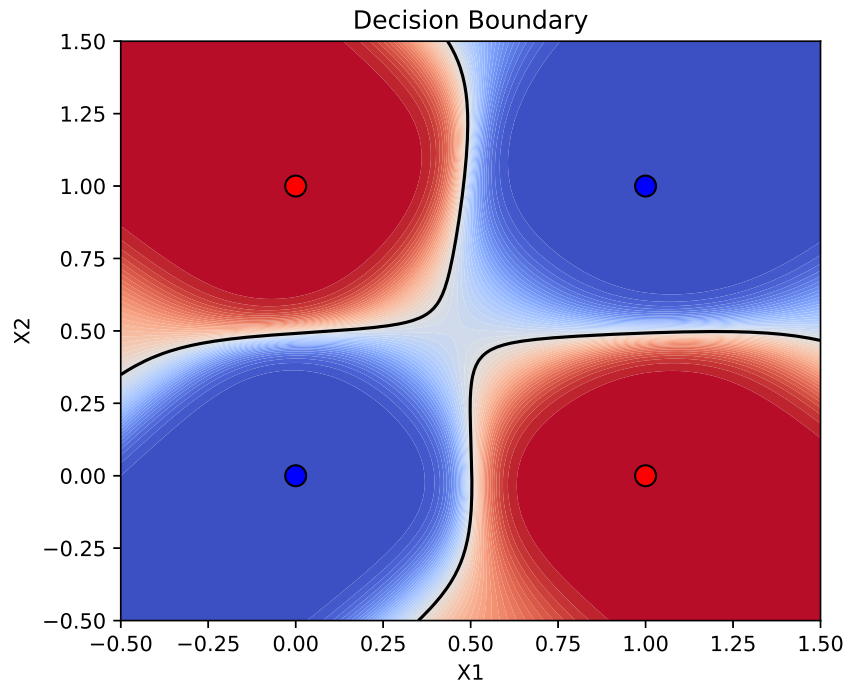


Figure 13: Decision boundary learned by the Multi-Layer Perceptron for the XOR problem.

6 Conclusion

The experiment demonstrated the limitation of the single-layer perceptron on the XOR problem and highlighted the capability of the Multi-Layer Perceptron to solve non-linearly separable classification tasks. By introducing a hidden layer and nonlinear activation functions, the network successfully learned the XOR relationship and achieved perfect classification accuracy. The accuracy, loss, and decision boundary plots confirmed the convergence and effectiveness of the implemented neural network.